SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. *Introduction:*

This document introduces **InvenIQ**, an Agentic Inventory Intelligence platform designed to solve the fragmented data crisis for Malaysian micro-SMEs (e.g., Kedai Runcit). By leveraging Groq's llama-3.3-70b-versatile, InvenIQ acts as a virtual CFO. It transforms raw operational data (CSV) and unstructured external signals (weather, news, market trends) into high-fidelity economic strategies, moving businesses from intuition-based to data-driven procurement.

1.1. Purpose:

The system analysis document highlights the technical scope, schema, and design-related decisions behind the development of the InvenIQ prototype.

The key elements of the system that have been covered are as follows:

1. ***Architecture**:*
    a. A Decoupled Proxy Architecture featuring a robust client-side pre-computation layer (Vanilla JS/HTML) and a secure middleware proxy (FastAPI/Python).
2. ***Data Flows:***
    a. Traces the journey from raw SME CSV data ingestion and client-side mathematical summarization, through the backend prompt-injection layer, to the Agentic JSON response rendered dynamically on the UI.
3. ***Model Process:***
    a. Defines the end-to-end workflow of the "Strategy Orchestration" engine, including the Zero-Data Fallback and Anti-Hallucination (Rule 8) protocols.
4. *Role of Reference:*
    a. Serves as a technical blueprint for all developers, data analysts, and testers to understand the high-level architecture and guardrails of this project.

***1.2. Background**:*

Currently, micro-SME owners in Malaysia rely heavily on manual "gut feeling" for inventory restocking. They lack the technical expertise and budget to utilize professional data analysts. Consequently, they lose an estimated 15-20% of potential revenue due to "Dead Stock" (unsold items) and "Out-of-Stock" scenarios, failing to account for external factors like sudden price hikes or weather-driven demand.

1. *Previous Version:*
    a. Existing solutions utilized by this target market are either manual spreadsheets which are highly error-prone and lacking predictive insight or expensive Enterprise Resource Planning (ERP) systems that do not possess the contextual "reasoning" capabilities

© UMHackathon 2026

required to correlate external events like a local festival or monsoon with historical sales data.

2. *Changes in Major Architectural Components*
   a. **Shift to a Decoupled Agentic Architecture**
      - **Previous State:** Traditional SME tools use monolithic architectures or basic spreadsheet software that process data locally but lack predictive reasoning.
      - **New Component:** InvenIQ introduces a lightweight, decoupled Client-Server Proxy Architecture. The heavy reasoning is offloaded to the cloud via Groq API (llama-3.3-70b-versatile), while the user only requires a lightweight web browser. The backend (FastAPI) acts as a secure middleware layer to prevent direct, unformatted access to the AI.
   b. **Introduction of Client-Side Deterministic Pre-Computation**
      - **Previous State:** Standard AI chatbots send entire raw datasets to the LLM, which consumes massive amounts of tokens, increases API costs, and often leads to mathematical errors (LLMs struggle with large-scale counting).
      - **New Component:** InvenIQ introduces a dedicated JavaScript Pre-processing Layer on the frontend. When a user uploads the sales_data_kaggle.csv, the client device deterministically calculates the Top 5 and Bottom 5 selling products before communicating with the AI. This major architectural addition guarantees mathematical accuracy and drastically reduces token payload sizes.
   c. **Shift from Text-Streaming UI to Dynamic JSON Rendering**
      - **Previous State:** Conventional LLM interfaces stream unstructured "walls of text" back to the user, which is difficult for a busy Kedai Runcit owner to read quickly.
      - **New Component:** The frontend architecture now includes a JSON Interceptor & Parser. The AI is architecturally constrained to output strict JSON data formats. The frontend intercepts this JSON and dynamically constructs color-coded (Red/Amber/Green) interactive Dashboard Cards.
   d. **Implementation of a Secure Middleware Constraint Vault**
      - **Previous State:** Basic prototypes often expose their AI system prompts on the frontend, making them vulnerable to prompt injection or behavioral drift.
      - **New Component:** InvenIQ utilizes a FastAPI backend as a security proxy. The core AI identity (MASTER_PROMPT) and the conversational memory array are securely injected on the server side, ensuring the system cannot be manipulated into discussing non-business topics.

3. *New capabilities that are only introduced in this*
   a. **Multi-Signal Contextual Correlation:** For the first time, SME owners can mathematically correlate their historical structured sales data (Units Sold, Inventory Level) with unstructured external signals. The system is newly capable of ingesting toggled variables—such as "Heavy

© UMHackathon 2026

Rain," "Labour Day," or "Competitor Price Hikes"—and instantly recalculating inventory reorder priorities based on these real-world constraints.

b. **Rule 8 Anti-Hallucination Protocol:** Unlike generic LLMs that might invent fake products to solve a problem (e.g., suggesting a user sell "Umbrellas" when it rains, even if the shop doesn't carry them), InvenIQ introduces strict Identity-Match Guardrails. The system is newly capable of cross-referencing the AI's output against the Product ID schema of the uploaded CSV, absolutely forbidding the recommendation of non-existent inventory.

c. **Zero-Data Fallback & Graceful Degradation:** The system introduces an automated safety capability where it refuses to hallucinate financial projections. If a user asks a highly analytical question (e.g., "What is my forecast for next week?") without uploading their sales dataset, the AI recognizes the missing dependency, halts the mathematical projection, and safely degrades to a "Data Required" fallback UI.

d. **Agentic Procurement Strategy Generation:** Instead of merely telling the user *what* happened in the past (descriptive analytics like a spreadsheet), InvenIQ introduces prescriptive, agentic capabilities. It tells the user exactly *what* to order, *how many* units to buy, and provides the *economic reasoning* behind the recommendation, acting as a virtual Chief Financial Officer for the micro-SME.

1.3. Target Stakeholder

Stakeholders	***System Interaction***	***Expectations***
Kedai Runcit Owners and Micro-SME Retailers in Malaysia.	*These users interact entirely with the frontend Agentic UI). They upload their historical data, toggle the context signals (e.g., Weather, Market News), and consume the color-coded JSON strategy cards to make daily purchasing decisions.*	*Reduced mental fatigue, lowered risk of "Dead Stock," and optimized cash flow without needing to learn complex ERP software or hire a data analyst.*
Store Managers, Cashiers, or Inventory Clerks	*Responsible for generating the raw data. They export the daily/weekly sales logs from standard Point of Sale (POS) systems or manual ledgers into the CSV format required for InvenIQ's ingestion.*	*Drastic reduction in manual labor and workplace friction. Instead of spending hours physically counting stock or doing manual math to figure out what sold best, clerks simply export the daily CSV. Furthermore, it removes the "blame game" for out-of-stock items, as procurement decisions are shifted to the AI's data-driven recommendations rather than human guesswork.*

© UMHackathon 2026

Local Suppliers, Wholesalers, and Distributors.	*Zero direct interaction with the system.*	*By empowering Kedai Runcit owners to order inventory based on predictive data (e.g., ordering umbrellas before the monsoon season starts) rather than reactive panic-buying, local supply chains become more stabilized and predictable.*
The InvenIQ Engineering Team (Backend Developers, Prompt Engineers, Data Analysts).	*This group manages the FastAPI middleware). They do not interact with the UI; instead, they monitor the Groq llama-3.3-70b-versatile API token usage, manage the server logs, update the master prompt to prevent prompt-injection attacks, and maintain the Rule 8 Anti-Hallucination guardrails.*	*A highly maintainable, secure, and cost-effective system architecture. Because of the client-side pre-computation, developers benefit from drastically reduced Groq API token costs. Additionally, the decoupled FastAPI proxy allows administrators to instantly update the master prompt guardrails on the server side without needing to take the frontend offline or force users to download updates.*

© UMHackathon 2026

2. System Architecture & Design

2.1. System Architecture Overview

InvenIQ operates on a lightweight, decoupled Client-Server architecture, enhanced by a Retrieval-Augmented Generation (RAG) and Agentic workflow. This hybrid approach optimizes both computational efficiency and token management.

- **Presentation & Pre-computation Layer (Frontend):** A responsive, dashboard-style interface built with HTML, CSS, and Vanilla JavaScript. Crucially, this layer acts as the initial "Data Cruncher," performing deterministic mathematical operations (e.g., sorting top/bottom sellers) and dynamic UI rendering based on structured JSON responses.

- **Application Proxy Layer (Backend):** A fast, asynchronous RESTful API powered by Python and **FastAPI** (groq_backend.py). It acts as a secure middleware orchestrator, managing cross-origin resource sharing (CORS) and routing requests seamlessly between the client and the AI engine.

- **Cognitive & Agentic Layer (LLM Engine):** The core decision-making unit powered by **Groq's llama-3.3-70b-versatile API**. Constrained by strict system prompts and zero-hallucination guardrails, it acts as an intelligent agent synthesizing structured CSV data with unstructured external signals.

2.2. System Logic and Data Flow

The system logic merges deterministic code with probabilistic AI to create highly accurate, business-ready insights. The data flows through the following pipeline:

1. **Client-Side Ingestion & Crunching:** The user uploads raw tabular data (sales_data.csv). Instantly, the JavaScript engine bypasses LLM token limits by mathematically calculating the exact Top 5 Best-Selling and Bottom 5 Worst-Selling products locally.

2. **Context Assembly:** The frontend compiles this mathematical output, along with a 30-row schema sample, into a hidden "Data Snapshot."

3. **Multi-Signal Integration:** The user inputs unstructured external signals (e.g., Calendar events like "Labour Day", Weather Outlooks like "Heavy Rain", or Market News) alongside their specific query.

4. **Prompt Orchestration:** The frontend constructs a robust payload containing: The Persona Instructions + JSON Formatting Rules + Anti-Hallucination Protocol (Rule 8) + The External Signals + The Data Snapshot. This is routed via the FastAPI backend to the Groq LLM.

© UMHackathon 2026

5. **Context-Aware Inference:** The Groq model correlates the unstructured signals with the structured Data Snapshot (e.g., recognizing that "Heavy Rain" reduces foot traffic but spikes demand for specific "hot beverages" already in the store's dataset).

6. **Agentic UI Rendering:** The AI outputs a highly structured JSON response. The frontend intercepts this JSON and dynamically renders beautiful UI cards displaying a Direct Answer, Status Badges (Green/Amber/Red), Trend Forecasts, and precise Restocking Actions.

2.3. System Schema and Database Design

While the architecture prevents sending massive raw datasets directly to the LLM, the underlying schema validation is critical for the AI's reasoning accuracy. The core schema vectors parsed into the "Data Snapshot" include:

- Product Identifiers: Product ID, Category (Crucial for the AI's Anti-Hallucination Protocol, ensuring it only recommends existing inventory).
- Deterministic Metrics (Calculated via JS): Units Sold, Inventory Level, Demand (Used to calculate Top/Bottom performers).
- Unstructured Variables: Weather, News, Promotions.

Design Logic (Anti-Hallucination): By feeding only the optimized Data Snapshot and enforcing "Rule 8" (strictly forbidding the naming of products outside the snapshot), the system guarantees that all trade-off analyses and recommendations are grounded entirely in the SME's actual business reality.

2.4. Technical Feasibility & Workflow Integration

This design is highly feasible, scalable, and tailored to the constraints of SME operations:

- Cost & Token Optimization: By utilizing client-side JavaScript for heavy mathematical sorting and data truncation, the system prevents the LLM from wasting tokens on tasks it struggles with (math), reserving the 8192 token limit purely for complex reasoning and generating the robust JSON output.
- Deterministic vs. Probabilistic Balance: Standard AI tools fail at business intelligence because they attempt to do math probabilistically. InvenIQ solves this natively in the frontend, ensuring quantifiable metrics are 100% accurate while leveraging the AI solely for strategic correlation.
- Seamless UI Experience: The strict JSON-to-UI-Card rendering transforms the tool from a standard "chatbot" into a true "Agentic Dashboard," making complex data readable for non-technical SME owners at a glance.

© UMHackathon 2026

3. Functional Requirements & Scope

3.1. In-Scope Functional Requirements (Core Features)

These requirements define the necessary behaviors of the InvenIQ prototype to fulfill the Decision Intelligence criteria of the hackathon domain.

- FR-01: Client-Side Data Ingestion & Mathematical Crunching
  - The system must allow users to upload a structured CSV file (e.g., retail sales data).
  - The frontend JavaScript engine must parse the dataset locally to bypass LLM token limits and eliminate AI mathematical errors.
  - The system must automatically calculate the exact Top 5 Best-Selling and Bottom 5 Worst-Selling products and package them into a hidden "Data Snapshot."
- FR-02: Multi-Signal Context Integration
  - The system must accept unstructured external signals via natural language input (e.g., "Heavy rain for the next 3 days" or "Upcoming Labour Day weekend").
  - The Groq LLM engine must correlate these unstructured external signals with the structured "Data Snapshot" to identify demand fluctuations and trade-off scenarios.
- FR-03: Agentic UI and JSON-to-Dashboard Rendering
  - The Groq LLM engine must be strictly prompted to output responses in a standardized JSON format.
  - The frontend system must intercept this JSON and dynamically render interactive UI Dashboard Cards. These cards must display:
    - A one-sentence Direct Answer.
    - Color-coded Status Badges (Green/Amber/Red) indicating stock health.
    - Expected unit sales forecasts.
    - Precise Recommended Actions (What to order, how many, when, and why).
- FR-04: Zero-Hallucination Guardrails (Rule 8 Enforcement)
  - The system must enforce a strict constraint on the Groq model, prohibiting it from inventing or recommending products that do not exist in the provided CSV Data Snapshot.
  - The AI must reference exact Product IDs and Categories supplied by the user's dataset.

3.2. Out-of-Scope (Scope Control)

To ensure a feasible Minimum Viable Product (MVP), the following features are explicitly excluded from the current prototype:

- **Automated Purchasing/Procurement:** The system acts as a *Decision-Support Tool*. It will recommend what to order and calculate the financial impact, but it will *not* automatically execute purchase orders with suppliers. Human approval remains the final step.
- **Live POS Database Integration:** For the preliminary round, the prototype relies on static CSV file uploads rather than establishing live API connections to enterprise Point-of-Sale (POS) systems (like SAP or Oracle).

© UMHackathon 2026

- **User Authentication & Persistent Login:** The current prototype focuses solely on demonstrating the AI's core reasoning and UI generation. Account creation, login screens, and persistent cloud saving of chat histories are deferred to post-MVP development.

3.3. Target User Group & Primary Use Cases

- **Target User:** SME Retail Owners, Inventory Managers, and Supply Chain Analysts in Malaysia.
- **Primary Use Case (Scenario-Based Validation):** An SME owner realizes a major Epidemic or sudden Weather Event (e.g., Monsoon season) is approaching.
  a. *Action:* The user uploads their current sales data and inputs the external signal into InvenIQ.
  b. *Value Realized:* Instead of guessing, the system instantly calculates the historical impact of similar events, checks current inventory levels, and renders an actionable card advising the owner to immediately increase stock of 'Groceries' by 20% to avoid stockouts, quantifying the potential revenue saved.

3.4. Non- Functional Requirements (NFRs)

While Functional Requirements focus on the Agentic UI and data analysis, the following NFRs ensure the system is enterprise-ready for Malaysian SMEs.

- NFR-01: Performance & Latency (Responsiveness)
  - The backend FastAPI orchestrator must ensure that the "typing indicator" and initial handshake occur in under 200ms.
  - Due to the complexity of the llama-3.3-70b-versatile reasoning, the final JSON-rendered dashboard should be delivered within 5 to 8 seconds, ensuring the user experience remains interactive without timing out.
- NFR-02: Accuracy & Reliability (Mathematical Integrity)
  - The system must maintain 100% deterministic accuracy for all "Top 5" and "Bottom 5" calculations. This is achieved by offloading math to the client-side JavaScript engine rather than relying on probabilistic AI calculations.
  - The AI's recommendations must adhere to the "Rule 8" protocol, ensuring a 0% hallucination rate regarding product names or IDs not found in the uploaded CSV.
- NFR-03: Security & Data Privacy (SME Protection)
  - As InvenIQ handles sensitive sales and pricing data, the prototype must ensure that data processed in the frontend remains in the volatile memory (RAM) of the user's browser.

© UMHackathon 2026

- The system must not store or persist the uploaded sales_data.csv on the backend server after the session is closed, mitigating the risk of data breaches for the SME.

- NFR-04: Scalability & Token Efficiency
  - The system must be optimized to handle CSV files up to 10,000 rows by utilizing the "Kitchen Sink" data-crunching method (summarizing data before sending it to the AI).
  - This ensures the system remains within the 1024 token limit, making it cost-effective and technically feasible for high-frequency use by SME owners.

© UMHackathon 2026

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Monitor, Verification and Resilience:

To ensure InvenIQ remains a reliable decision-support tool, the following monitoring and resilience strategies are implemented:

- **System Monitoring:** The FastAPI backend provides real-time console logging for all API requests and model inference cycles. Error handling is centralized; specifically, the groq_backend.py script captures Exception as e during the Groq API call, ensuring the server does not crash if the Groq service experiences latency.
- **Response Verification:** Before the frontend renders the Agentic UI, it performs a JSON Schema Validation. Since the system relies on the Groq LLM to output structured JSON for the dashboard cards, the frontend verifies the presence of required keys (e.g., direct_answer, recommended_action). If the AI fails to produce valid JSON, the system triggers a fallback "Text Rendering" mode to maintain usability.
- **Operational Resilience:** The "Master Prompt" serves as a continuous verification layer. It constantly monitors the user's intent; if a query deviates from business intelligence or inventory management, the AI is programmed to steer the conversation back, preventing the system from being used for unauthorized or non-productive purposes.

4.2. Assumptions:

The successful deployment and operation of the InvenIQ prototype are based on the following key assumptions:

- **Stable Connectivity:** Users (SME owners) possess a stable internet connection to facilitate the asynchronous handshake between the frontend and the Groq LLM hosted on GroqGroq's servers#x2019;s servers.

- **Data Consistency:** It is assumed that the uploaded CSV data follows a consistent schema (e.g., Product ID, Units Sold, Inventory Level).

- **Human-in-the-Loop:** The system is built on the assumption that it acts as an **Advisory Tool**. Final financial decisions and stock orders require human verification and approval before execution.

© UMHackathon 2026

4.3. External Dependencies:

The system's functionality is contingent upon the following external integrations:

Tool	Purpose	Risk	Mitigation
Groq LLM (llama-3.3-70b-versatile)	The core reasoning engine for interpretation and trade-off analysis.	High: API downtime or rate limiting would disable the decision intelligence.	Implementation of graceful degradation (notifying the user of service status).
FastAPI Framework	Acts as the backend microservice to manage prompts and API routing.	Low: Minimal risk unless the hosting environment (e.g., Render/AWS) fails.	Use of lightweight, stateless containers for rapid redeployment.
Groq Python SDK	The client library used to communicate with the Groq API.	Medium: Version mismatches or breaking changes in the SDK.	Locking dependency versions in a requirements.txt file.
Vanilla JavaScript (ES6+)	Handles client-side mathematical crunching of the CSV data.	Low: Compatibility issues with very old web browsers.	Ensuring the UI is optimized for modern browsers (Chrome/Edge/Safari).

4.4 Evaluation Metrics

- **Technical Accuracy:** Target a 100% match between JS-calculated "Top Sellers" and the data snapshot provided to the AI.
- **Inference Speed:** Successful rendering of the Agentic Dashboard within <8 seconds to maintain user engagement.
- **Economic Value:** Evaluation is based on the system's ability to identify at least two high-risk stockout scenarios per data upload, potentially saving the user the cost of lost sales.

5. Project Management & Team Contributions

The development of InvenIQ followed a compressed 10-day Agile sprint, focusing on delivering a high-impact Minimum Viable Product (MVP) that showcases Groq's decision-making capabilities.

5.1. Project Timeline:

Day 1–2: Domain Alignment & Data Exploration

- *Analyzed the sales_data_kaggle.csv to identify key decision triggers (Weather, Promos, Seasonality).*
- *Defined the "Decision Intelligence" scope: moving from simple automation to strategic advisory.*

Day 3–4: Data Engineering & Mathematical Logic

- *Developed the JavaScript pre-processor to handle deterministic calculations to ensure 100% accuracy.*
- *Drafted the "Data Snapshot" schema to optimize token usage.*

Day 5–6: Backend Architecture & Groq Integration

- *Built the FastAPI microservice.*

© UMHackathon 2026

- *Engineered the Master Prompt and Rule 8 (Anti-Hallucination) to guide the llama-3.3-70b-versatile reasoning.*

Day 7–8: Agentic UI Development & JSON Mapping

- *Developed the frontend dashboard).*
- *Mapped the AI's JSON output to dynamic, color-coded UI cards for immediate user action.*

Day 9–10: System Validation & Documentation

- *Conducted scenario-based testing (e.g., simulating the impact of a Monsoon on Grocery stock).*
- *Finalized the documents and Pitch Deck.*

5.2. Team Members Role:

Andrew Tee Kai Yang **– Role: Data Analyst:** Responsible for dataset selection, defining the multivariate schema, and designing the logic for the client-side deterministic math pre-processing.

Puah Khai Xin **– Role: Backend Lead:** Developed the FastAPI backend (groq_backend.py) and managed the secure integration of the Groq llama-3.3-70b-versatile model. Optimized the asynchronous communication between the frontend and the AI engine to handle high-token payloads without latency issues.

Harry Tan Hao Ran **– Role: Frontend/UI Lead:** Built the responsive InvenIQ Dashboard and the "Agentic UI" rendering engine. Coded the client-side JavaScript math engine that performs deterministic sorting and calculations, ensuring the UI cards reflect 100% accurate data before the AI provides its reasoning.

Nishaan Kumaar A/L Subramaniam **– Role: Prompt Engineer:** Crafted and refined the Master Prompt and Rule 8 (Anti-Hallucination Protocol). Conducted iterative prompt-tuning to ensure the model consistently outputs perfect JSON for the dashboard and maintains a professional, business-centric persona without deviating from the inventory domain.

Lian Xiao Wei **– Role: Product Manager:** Oversaw the PRD creation, aligned the technical architecture with the Economic Empowerment domain, and managed the QA testing (Zero-Data Fallback tests).

5.3. Recommendations:

To transition InvenIQ from a prototype to a production-ready solution for the Malaysian SME market, we recommend the following:

- Vector Database: Implement a vector database (e.g., Pinecone) to allow the AI to query years of historical sales data without hitting token limits, enabling long-term trend forecasting.

- Persistence Layer: Integrate a PostgreSQL database to store user chat histories and previous "Decision Cards," allowing the AI to learn from the owner's past preferences.

- Automated Forecasting Models: Complement the Groq LLM with a dedicated time-series forecasting model (like Prophet or ARIMA) to provide the AI with even more precise "baseline" numbers before it performs contextual reasoning.

© UMHackathon 2026

- Regional Localization: Fine-tune the context triggers for the Malaysian market, specifically including "Public Holiday" calendars (Hari Raya, CNY, Deepavali) as high-weight variables for demand prediction.

© UMHackathon 2026